



OPEN TECHNOLOGY GROUP (OTG)

National Informatics Centre

PageIndex

RAG System

AI-Powered PDF Question Answering Platform

PDF

Embedding

FAISS

AI Answer

Fast • Accurate • Offline AI

TABLE OF CONTENTS

01	Executive Summary
02	Project Overview
03	Objectives
04	Technology Stack
05	System Workflow
06	Enterprise Architecture
07	Module Explanation
08	RAG Pipeline Explanation
09	FAISS Vector Database Workflow
10	Ollama Integration
11	API Flow Architecture
12	Security Architecture
13	Deployment Architecture
14	Monitoring and Logging
15	Performance Optimization
16	Folder Structure
17	Future Enhancements
18	Final Output / Result Conclusion
19	Conclusion

01 EXECUTIVE SUMMARY

This project is an Enterprise-grade PDF-based RAG (Retrieval-Augmented Generation) AI chatbot system developed using Python, Flask, FAISS Vector Database, and Ollama Local LLM.

The System Allows Users To

- ▶ Upload PDF documents
- ▶ Convert PDF content into embeddings
- ▶ Store embeddings in FAISS vector database
- ▶ Perform semantic and keyword-based retrieval
- ▶ Generate accurate AI responses from uploaded PDFs only
- ▶ Prevent hallucinations using strict validation logic

The Solution Is Optimized

- ▶ High-speed retrieval
- ▶ Offline local AI execution
- ▶ Secure enterprise environments
- ▶ Accurate document question answering
- ▶ Lightweight deployment

The Enterprise-grade PDF RAG AI Chatbot Executive Summary Content

- ▶ Outline key features and benefits of the PDF RAG system.
- ▶ Describe technical architecture and component interactions succinctly.
- ▶ Highlight security, offline, and enterprise deployment advantages.
- ▶ Draft a concise executive summary and next-step recommendations.
- ▶ Provide usage scenarios and example user workflows

02 PROJECT OVERVIEW

The project implements a complete Retrieval-Augmented Generation (RAG) architecture. PageIndex RAG is designed to answer questions directly from uploaded PDF documents, combining semantic vector search using FAISS with local Large Language Models running through Ollama.

Core Functionalities

Feature	Description
PDF Upload	Upload enterprise documents
PDF Parsing	Extract text using PyPDF
Smart Chunking	Split documents into meaningful chunks
Embedding Generation	Create vector embeddings using Ollama
FAISS Storage	Store embeddings in vector database
Semantic Search	Retrieve relevant chunks
Raw Search	Exact phrase and keyword matching
AI Response Generation	Generate context-aware answers
Validation Layer	Prevent invalid AI responses
REST API	Flask API for frontend communication

03 OBJECTIVES

Primary Goals

- Build a lightweight enterprise AI assistant
- Achieve fast PDF-based search retrieval
- Minimize hallucination issues
- Support local offline AI execution
- Create secure document intelligence platform
- Provide accurate contextual answers

Business Benefits

Benefit	Impact
Faster Document Search	Reduces research time significantly
Reduced Manual Effort	Automated document Q&A
Secure Local Processing	No cloud data exposure
Enterprise Knowledge Automation	Scale AI across departments
Scalable AI Architecture	Grows with organizational needs

04 TECHNOLOGY STACK

Technology	Purpose
Python	Core backend language
Flask	Web framework and API server
FAISS	High-speed vector similarity search
Ollama	Local LLM inference engine
nomic-embed-text / mxbai-embed-large	Embedding model
llama3.2:3b / Mistral	Lightweight response generation model
NumPy	Vector processing
Requests	API communication
PyPDF	PDF text extraction
HTML/CSS/JS	Frontend UI

Why These Technologies?

FAISS — Ultra-fast vector similarity search; scales to millions of embeddings.

Ollama — Runs AI models 100% locally; zero cloud cost, full data privacy.

nomic-embed-text — State-of-the-art semantic understanding for retrieval quality.

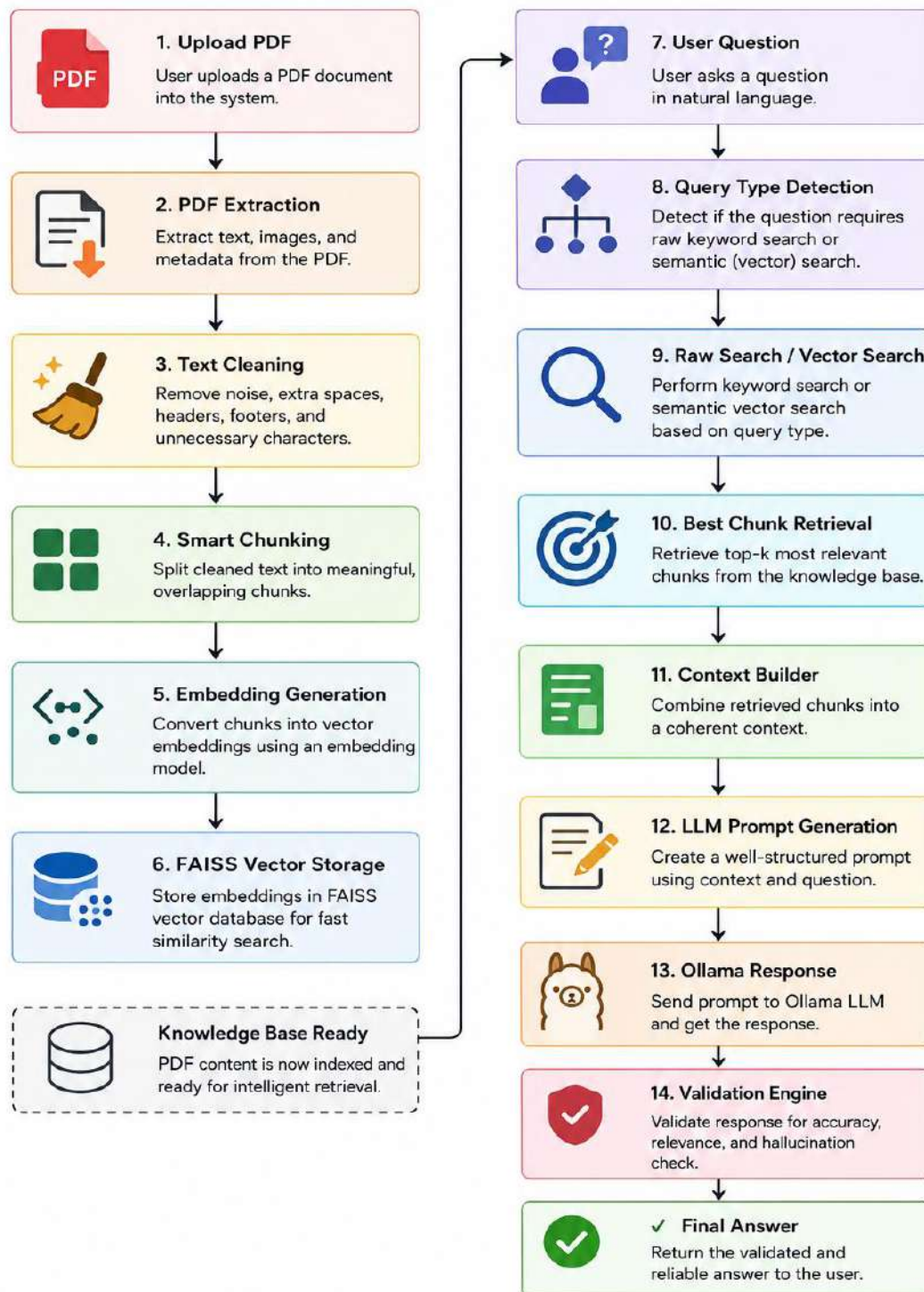
Mistral / llama3.2:3b— Fast, low-latency response generation with minimal resources.

Flask — Lightweight backend APIs for upload and chat functionality.

05 SYSTEM WORKFLOW

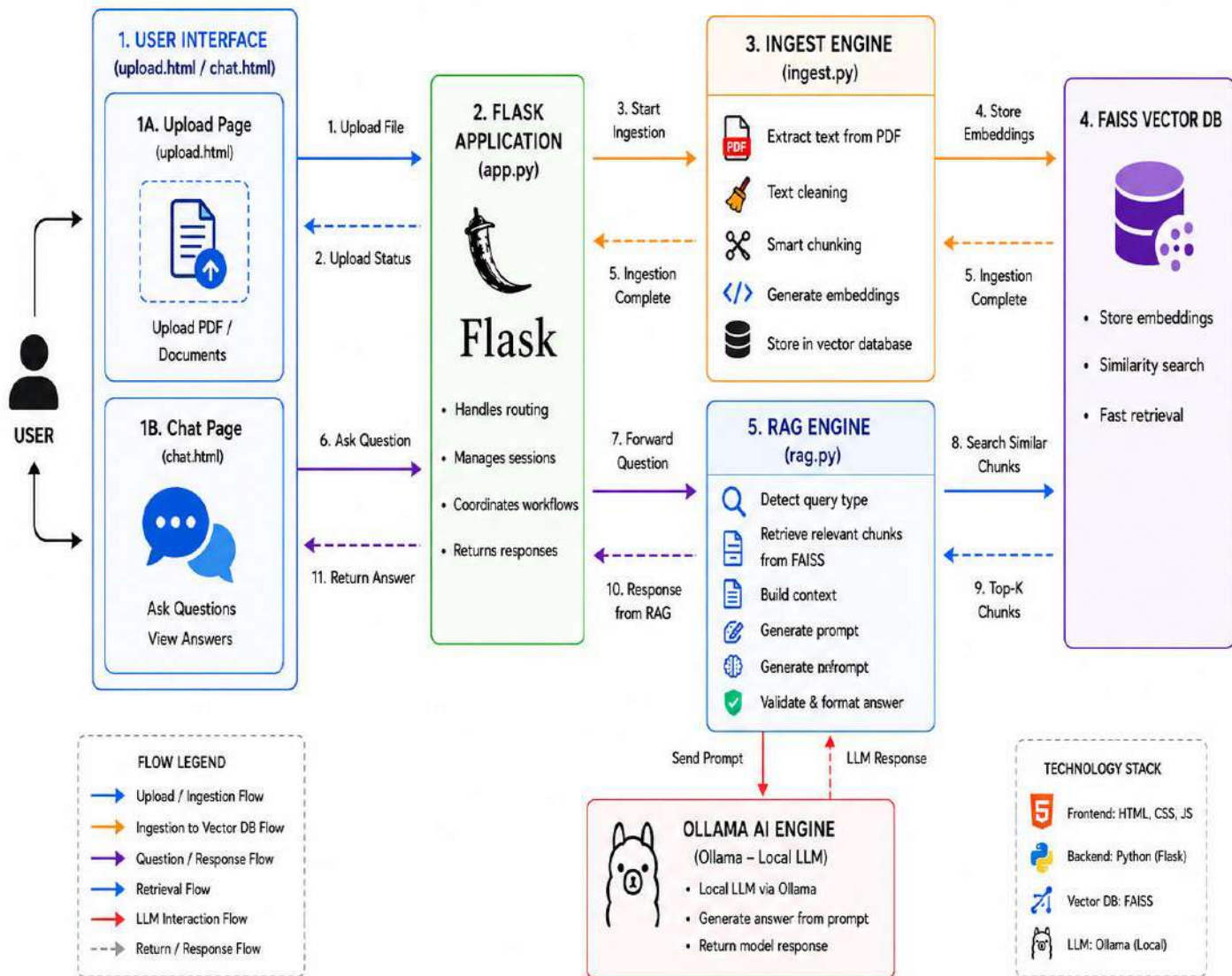
The complete end-to-end workflow from PDF ingestion to AI answer delivery:

PDF QA Workflow



06 ENTERPRISE ARCHITECTURE

RAG SYSTEM ARCHITECTURE



07 MODULE EXPLANATION

- ▶ This project is built using a Flask-based backend where app.py acts as the main controller for handling user requests. When a user uploads a PDF through the /upload_pdf endpoint, app.py forwards the file to ingest.py, which processes the document and builds a FAISS vector index from the extracted text.
- ▶ For user queries, the /chat endpoint in app.py sends the input to rag.py, where the RAG pipeline is executed. The pipeline follows multiple stages including Direct QA, Exact Entry matching, Section-based retrieval, Vector similarity search using FAISS, and finally an LLM fallback for generating accurate responses when retrieval is insufficient.
- ▶ Overall, ingest.py is responsible for preparing and indexing PDF data, while rag.py handles intelligent retrieval and response generation, making the system an efficient PDF-based AI chatbot.

Module	File	Responsibility
API Server	app.py	Flask REST API — upload and chat endpoints
PDF Ingestion	ingest.py	PDF parsing, chunking, and embedding generation
RAG Engine	rag.py	Vector search, keyword search, and LLM answer generation
Frontend	templates/	HTML/CSS/JS chat and upload UI

Module Interaction Flow

1. app.py receives /upload_pdf → calls ingest.py → FAISS index is built.
2. app.py receives /chat query → calls rag.py ask() → returns JSON answer.
3. rag.py runs Direct QA → Exact Entry → Section → Vector → LLM fallback pipeline.
4. ingest.py → builds FAISS index from PDFs

08 RAG PIPELINE EXPLANATION

Stage	Step	Description
Ingestion	1. Extract	pdfplumber extracts raw text and tables from PDF pages
Ingestion	2. Section Split	Regex detects UPPERCASE headers to split document sections
Ingestion	3. Chunk	Q/A, numbered, bullet, and plain section chunkers run per section
Ingestion	4. Embed	Ollama generates vector embeddings for each chunk
Ingestion	5. Index	FAISS IndexFlatIP stores L2-normalised vectors
Retrieval	6. Query Embed	User query is embedded using the same model
Retrieval	7. Vector Search	Top-k nearest chunks retrieved by cosine similarity
Retrieval	8. Keyword Boost	Keyword-scored chunks added to candidate pool
Answer	9. QA Extraction	Regex extracts Q/A pair answers from candidate chunks
Answer	10. LLM Generation	Ollama LLM generates final answer from context window
Answer	11. Validation	Not-found patterns rejected; hallucinations filtered

09 FAISS VECTOR DATABASE WORKFLOW

Phase	Action
Index Build	Convert each chunk to float32 embedding vector
Index Build	L2-normalise all vectors (faiss.normalize_L2)
Index Build	Add to IndexFlatIP (inner product = cosine on normalised vectors)
Index Build	Persist as index.faiss + chunks.txt
Query	Embed user query with same model
Query	L2-normalise query vector
Query	Search top-k by inner product similarity (≥ 0.25 threshold)
Query	Merge vector results with keyword-scored candidates

Embedding Model

Model	Purpose
nomic-embed-text	Semantic embedding for retrieval
mxbai-embed-large	Alternative high-quality embedding model

10 OLLAMA INTEGRATION

Endpoint	Port	Purpose
api/embeddings	11434	Generate vector embeddings for chunks and queries
api/generate	11434	Generate natural language answers from context

LLM Model

Model	Purpose
llama3.2:3b	Generate AI responses (lightweight, fast)
Mistral	Alternative fast local answer generation model

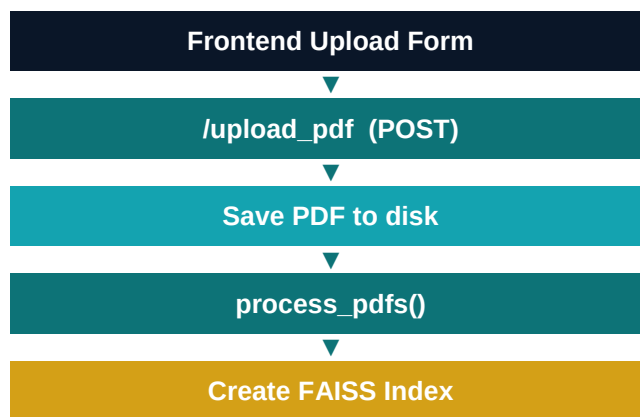
Advantages of Local Ollama Execution

- Fully local execution — zero cloud API cost
- No external data dependency or network latency
- Secure for sensitive enterprise documents
- Faster inference on local hardware
- Supports air-gapped / offline deployment

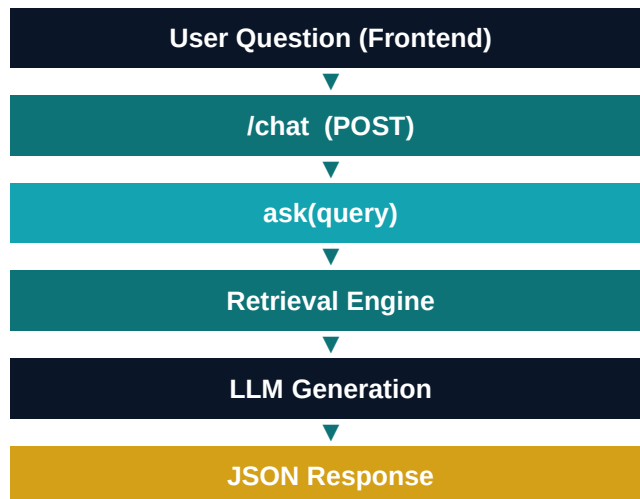
11 API FLOW ARCHITECTURE

The system is built using a Flask-based API architecture that handles both PDF ingestion and user interaction through separate workflows. In the **Upload API Flow**, the user uploads a PDF file from the frontend form, which is sent to the `/upload_pdf` endpoint using a POST request. The backend saves the file to disk and triggers the `process_pdfs()` function, which extracts and processes the document content. After processing, the system generates embeddings and builds a FAISS index for efficient semantic search and retrieval.

Upload API Flow



Chat API Flow



12 SECURITY ARCHITECTURE

The system is designed with a strong focus on data privacy and controlled AI behavior, ensuring secure processing of sensitive documents within a local environment. All AI operations are performed using **local model execution**, preventing any cloud-based data leakage and keeping information fully on-premise. Context validation mechanisms are applied so that responses are strictly generated from uploaded PDF content, reducing hallucination and ensuring factual accuracy

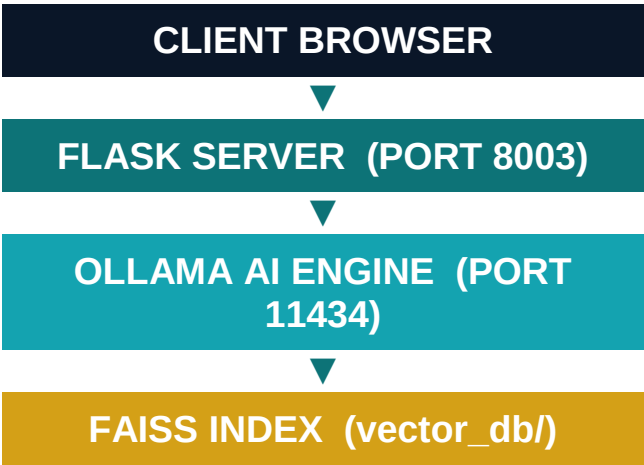
Security Feature	Purpose
Local AI Execution	Prevent cloud data leakage — all processing on-premise
Context Validation	Prevent hallucination — answers sourced from PDF only
Restricted Prompting	LLM instructed to avoid outside knowledge
Controlled Retrieval	Only PDF chunks used as context — no web lookups
API Isolation	Flask API runs on internal network port only

Enterprise Security Benefits

- ▶ Sensitive document protection — data never leaves the server.
- ▶ Offline deployment support — fully operational without internet.
- ▶ Internal network compatible — no firewall exceptions required.
- ▶ Reduced external attack surface — no third-party AI API keys.

13 DEPLOYMENT ARCHITECTURE

The system follows a lightweight local deployment architecture where the client interacts through a browser interface, which communicates with a Flask backend server running on port **8003**. The Flask server acts as the central processing layer, handling both PDF ingestion and user query requests. For AI-based response generation, it connects to the **Ollama AI engine** running locally on port **11434**, which processes prompts and generates context-aware outputs. The system also relies on a **FAISS vector database** stored in the `vector_db/` directory for fast semantic search and retrieval of relevant PDF chunks.



Production Hardware Requirements

Component	Recommended Spec
RAM	16 GB+
CPU	8 Cores
GPU	Optional (accelerates Ollama inference)
Storage	SSD (fast FAISS index load)
OS	Linux / KaliLinux

14 MONITORING AND LOGGING

The system includes a built-in logging mechanism to ensure better operational visibility and easier debugging of the RAG pipeline. It records important runtime events such as user queries, the type of retrieval used (vector search, keyword match, or direct QA), retrieved context chunks, raw responses generated by the Ollama model, embedding-related errors, and any issues encountered during PDF upload and processing. This helps in tracking system behavior and identifying performance or data-related issues.

Current Logging

The system already logs the following events for operational visibility:

- User queries
- Retrieval type (vector / keyword / QA)
- Context chunk output
- Raw AI response from Ollama
- Embedding errors
- PDF upload errors

Recommended Enhancements

Tool	Purpose
Structured Logging	JSON-formatted logs for machine parsing
ELK Stack	Centralized log aggregation and search
Prometheus	Metrics collection (latency, error rate)
Grafana	Visual dashboards for real-time monitoring
Request Tracing	End-to-end query trace IDs

15 PERFORMANCE OPTIMIZATION

The system incorporates multiple optimization techniques to improve retrieval accuracy, response speed, and overall AI performance. Embedding normalization is applied to enhance cosine similarity calculations during vector search. Query type detection helps route user queries through the fastest and most suitable retrieval path, improving efficiency. A raw search priority mechanism is used to accelerate exact phrase matching, while a vector similarity threshold of **0.25** helps filter out irrelevant results and reduce retrieval noise. Additionally, chunk filtering ensures that only highly relevant context is passed to the LLM, improving response quality and reducing unnecessary token usage. A validation layer is also included to minimize hallucinated outputs after response generation.

Optimizations Implemented

Optimization	Benefit
Embedding Normalization	Better cosine similarity accuracy
Query Type Detection	Routes queries to fastest retrieval path
Raw Search Priority	Improved exact phrase matching speed
Vector Similarity Threshold (0.25)	Noise reduction in retrieved chunks
Chunk Filtering	Only high-relevance chunks passed to LLM
Validation Layer	Reduces hallucination post-generation

Planned Future Optimizations

- Async PDF processing for large document batches
- GPU acceleration for Ollama inference
- Redis embedding cache for repeated queries
- Multi-threaded embedding generation
- Incremental FAISS index updates
- Hybrid reranking with cross-encoder models

16 FOLDER STRUCTURE

```
project/
├── app.py           ← Flask API server
├── ingest.py        ← PDF ingestion & FAISS indexing
├── rag.py           ← Retrieval & answer generation
├── data/
│   └── [uploaded PDFs]
├── vector_db/
│   ├── index.faiss  ← FAISS vector index
│   └── chunks.txt   ← Raw chunk text store
├── templates/
│   ├── upload.html
│   └── chat.html
└── static/
    └── [CSS / JS assets]
```

The project is organized using a modular folder structure to separate API handling, PDF processing, retrieval logic, frontend templates, and vector database storage. This structure improves maintainability, scalability, and easier deployment. The following section provides a detailed explanation of the project folder structure, describing the purpose and functionality of each file and directory used in the RAG-based AI chatbot system.

app.py

The main Flask application file that acts as the API server. It handles frontend requests such as PDF uploads and chat queries, manages API routes, and connects the frontend with the ingestion and retrieval modules.

ingest.py

Responsible for the PDF ingestion pipeline. It extracts text from uploaded PDF files, performs text cleaning and chunking, generates embeddings, and builds the FAISS vector index used for semantic retrieval.

rag.py

Implements the Retrieval-Augmented Generation (RAG) pipeline. It processes user queries, performs keyword and vector-based retrieval, validates context, and generates AI responses using the Ollama LLM.

data/

Stores all uploaded PDF documents. These files act as the primary knowledge source for the chatbot and are processed during ingestion.

Contents:

- Uploaded PDF files
 - Source documents for indexing
-

Vector_db

Contains the vector database and processed text chunks used for semantic search and retrieval.

Files:

- `index.faiss` → Stores FAISS vector embeddings
 - `chunks.txt` → Stores chunked text extracted from PDFs
-

Templates/

Contains frontend HTML templates used for the web interface.

Files:

- `upload.html` → PDF upload interface
 - `chat.html` → Chatbot interaction page
-

Static/

Stores static frontend assets required by the web application.

Contents:

- CSS stylesheets
- JavaScript files
- Images or UI resources

17 FUTURE ENHANCEMENTS

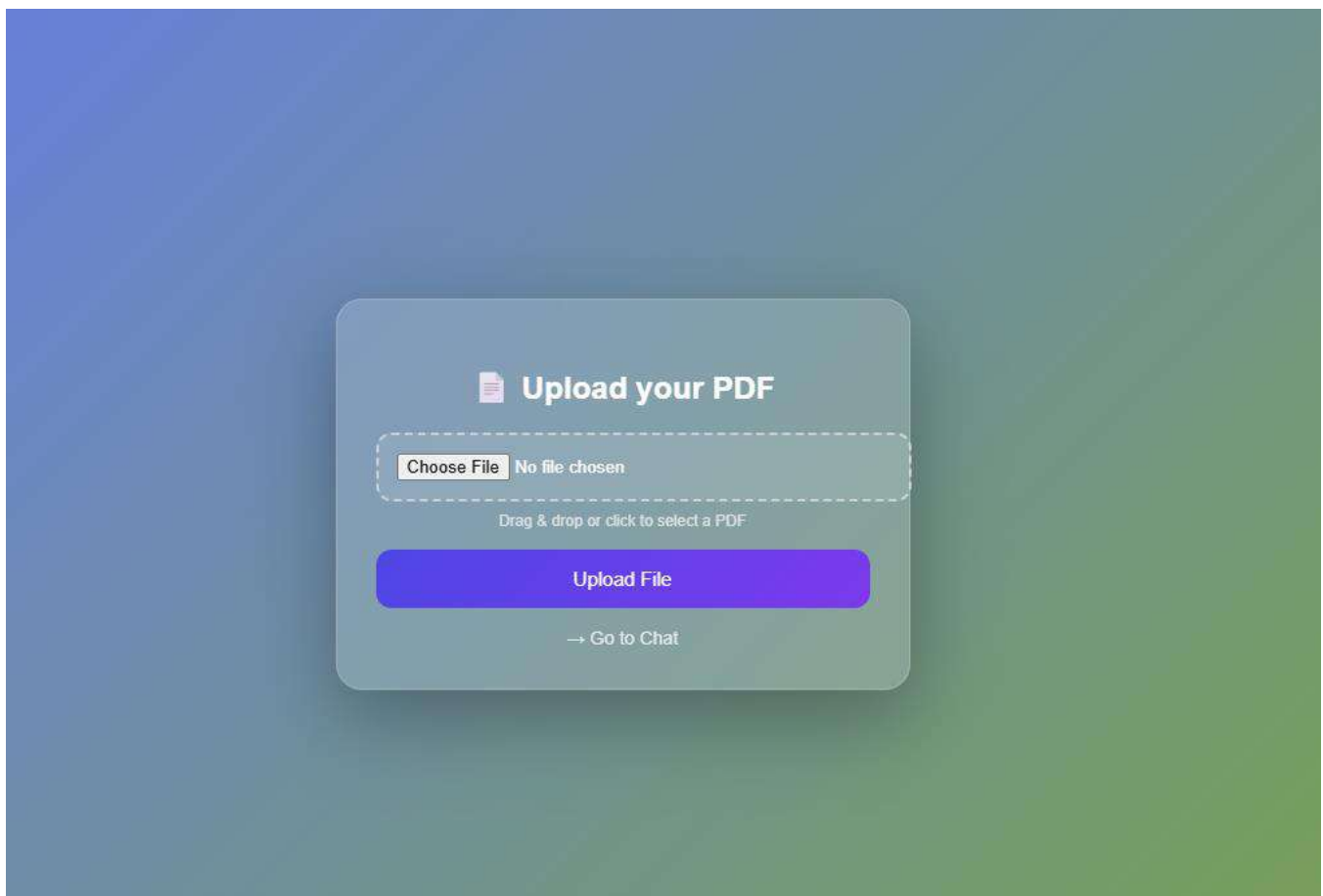
The system is designed with a scalable architecture that supports future expansion and advanced enterprise features. Several enhancements are planned to improve security, usability, intelligence, retrieval quality, and deployment flexibility. Authentication mechanisms such as multi-user login and role-based access control (RBAC) can be integrated to support secure enterprise access management. OCR capabilities are also planned to enable processing of scanned and image-based PDF documents that currently cannot be extracted using standard text parsing methods.

To improve accessibility and user interaction, future versions may include multilingual embedding and response generation support, along with voice-based chatbot interaction. Additional AI intelligence features such as automatic document summarization and conversation memory across sessions are planned to provide more contextual and personalized responses. Retrieval accuracy can be further enhanced using advanced reranking techniques with cross-encoder models.

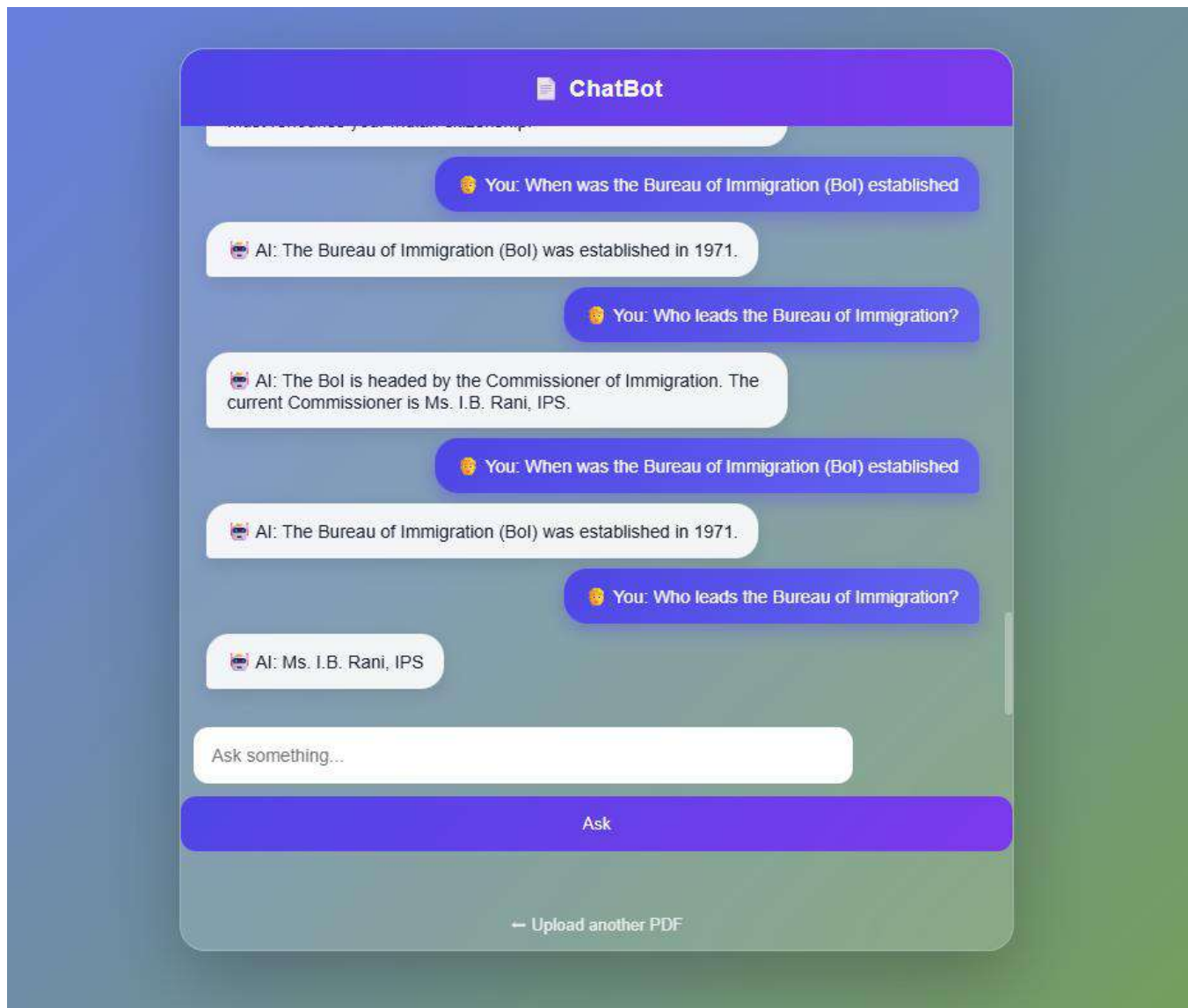
Category	Enhancement
Authentication	Multi-user login with role-based access control (RBAC)
Document Support	OCR integration for scanned / image PDFs
Multilingual	Multi-language embedding and response support
Interface	Voice chatbot interface integration
Intelligence	Document summarization module
Intelligence	Conversation memory across sessions
Retrieval	Advanced reranking with cross-encoder models
Infrastructure	Kubernetes container orchestration deployment
Infrastructure	API gateway integration for enterprise SSO

18 Final Output / Results

The attached screenshots illustrate the frontend upload interface, successful PDF ingestion workflow, chatbot interaction window, and generated responses based on document retrieval. These outputs confirm that the system operates correctly and efficiently in a local deployment environment.



The chatbot conversation screenshots show how the system accepts user queries, retrieves relevant information from the uploaded PDF, and generates accurate context-aware responses using the implemented RAG pipeline. These outputs confirm the successful integration of PDF ingestion, semantic retrieval, and AI-based response generation within the application.



19 CONCLUSION

PageIndex RAG provides a fast and accurate AI document question-answering system using local embeddings, FAISS vector search, and lightweight LLMs. The system supports fully offline operation and delivers precise, validated answers from uploaded PDF documents.

Architecture Summary

Layer	Technology	Role
Ingestion	pdfplumber + Chunkers	Extract, clean, and split PDF content
Embedding	Ollama + nomic-embed-text	Generate semantic vector representations
Storage	FAISS IndexFlatIP	Millisecond-speed vector similarity search
Retrieval	Vector + Keyword Hybrid	High-recall relevant chunk retrieval
Generation	Ollama + Mistral/phi3	Context-grounded answer generation
Validation	Pattern matching + threshold	Hallucination prevention

Suitable Deployment Contexts

- ▶ Government organizations and public sector bodies
- ▶ Enterprise knowledge management systems
- ▶ Research and academic document search platforms
- ▶ Legal and compliance document Q&A
- ▶ HR policy and internal procedure assistants
- ▶ Secure internal AI portals

System Architecture Flow:
PDF → Text Extraction → Chunking → Embeddings → FAISS DB
→ Retrieval → LLM → Answer

